



Høyskolen
Kristiania

Dato: 11.11 – 02.12.2025

Kandidatnr.: 8, 46, 75

Gruppenummer: 19

Eksamensrapport

JIIKAN API app

Emnekode: PGR208

Emnenavn: Android programmering

Termin: 2025 Høst

Rapport

Del 1. Redegjørelse av app per skjerm

Generelt

Navigation (AppNavigation / NavRoutes)

AppNavigation bruker Jetpack Compose for å skape en effektiv navigasjon i appen, og benytter RememberSaveable for å holde styr på aktiv skjerm. NavController navigerer mellom skjermene, mens NavHost definerer tilgjengelige skjermer. NavigationBar og NavigationBarItem gjør det enkelt for brukeren å bytte skjerm, og navBarTheme() sikrer et konsistent utseende. toRoute sender data mellom skjermene, noe som gir en brukervennlig navigasjonsopplevelse.

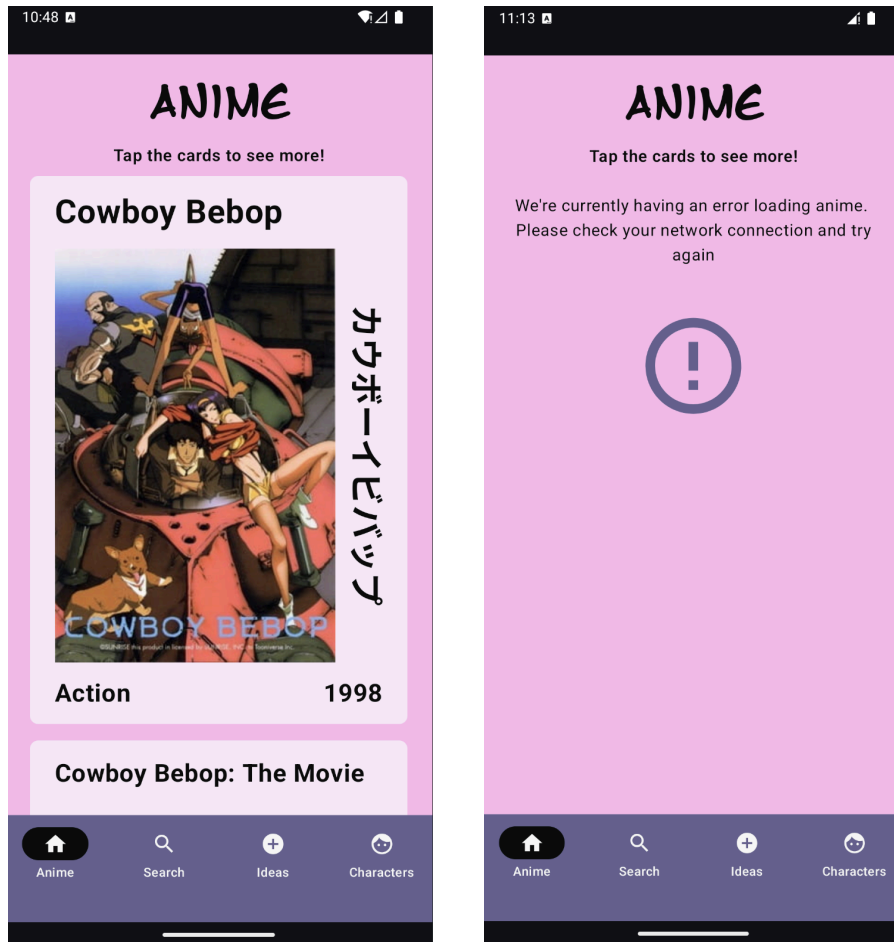
ViewModel

Hver skjerm har en egen ViewModel for å håndtere UI-relatert data og skjermens tilstandsendringer. ViewModelene eksponerer en offentlig kopi av state-variabler som skjermene kan observere, og oppdaterer state ved å hente data fra repository og håndtere brukerinteraksjoner. MutableStateFlow og .asStateFlow() brukes for å sende oppdateringer til UI-komponentene, noe som sikrer en robust applikasjon.

Komponenter

Vi har utviklet en rekke komponenter for å sikre konsistent design og gjenbrukbarhet. I tillegg til komponenter for ulike items (produktbokser) og List, har vi opprettet en "shared"-mappe med felles komponenter for farger, titler, undertitler, feilmeldinger, tekstfelt, brødtekst og andre gjentakende elementer. Dette bidrar til skalerbarhet og kortere kode, i tråd med god kodepraksis.

AnimeScreen



AnimeScreen viser informasjonskort for animer hentet fra Jikan APllet, som har 1177 sider med 25 animer per side (Jikan API, u. å. -a). For å sikre rask innlasting, viser vi kun én side. Ved problemer med API-innhenting, vises en feilmelding. Ved vellykket innhenting, vises informasjonskort med hovedinformasjon og bilde av hver anime.

Hvert kort har følgende verdier:

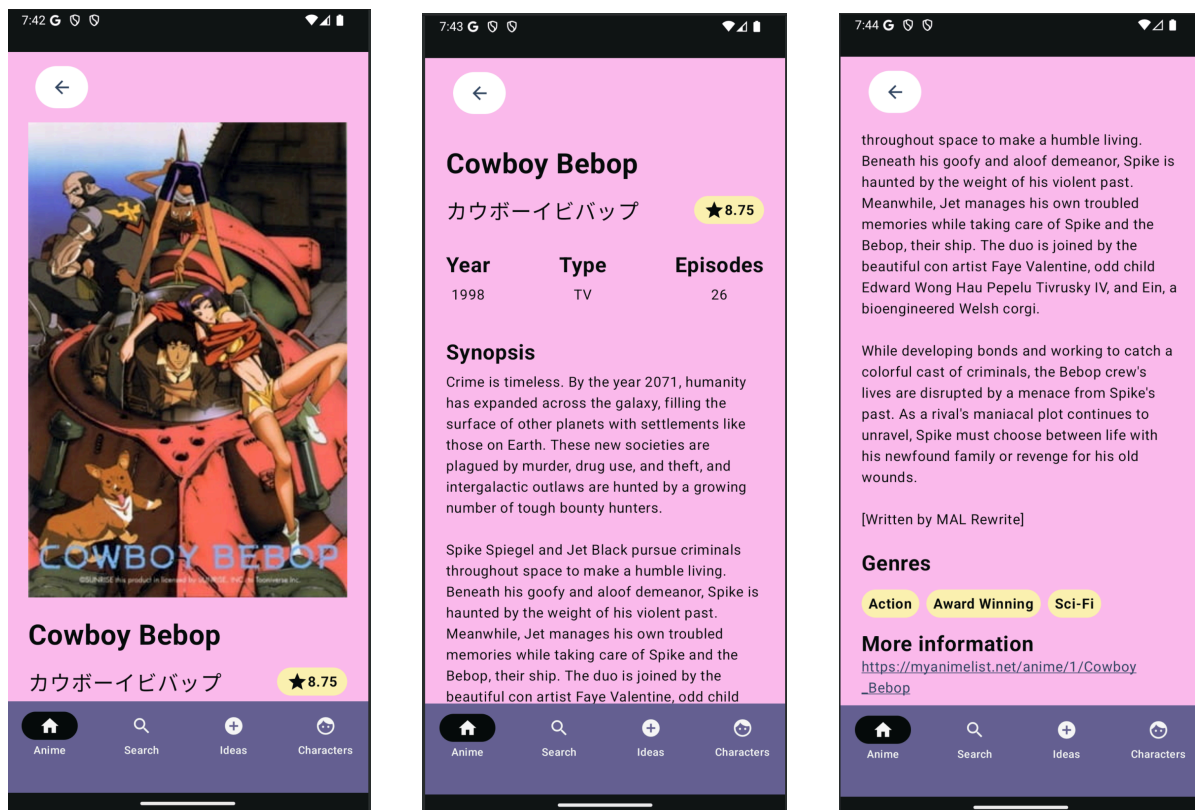
- **tittel** er hentet fra "title" fra APllet og satt "No title" på de som mangler tittel.
- **Japansk tittel** er hentet fra "title_japanese" fra APllet. Tittel blir satt til "No japeeese title" dersom objektet ikke har satt tittel på japansk. Vi ser at APllet noen ganger har skrevet tittelen på engelsk i title_japanese, men har valgt å følge deres oppsett på dette
- **Sjanger** fra listen "genres". Vi har valgt å holde oss til en sjanger i Anime siden for å holde informasjonskortet enkelt og kortfattet.

- **Utgivelsesår** er hentet fra “year” i APllet og viser utgivelsesåret til anime
- **Bilde** er hentet fra “images” i APllet. Images returnerer en liste med “jpg” og “webp” som viser til hvilken filtype som er brukt. Vi har valgt å bruke “jpg” som inneholder en liste med bilder i ulike størrelser. Vi har valgt å bruke “image_ur” da bildene vi viser er ganske store på skjermen.

AnimeScreen er koblet opp mot en ViewModel som kaller på AnimeAPIRepository sin getAllAnime metode og returnerer disse til ViewModel. getAllAnime kjører et GET kall opp mot Jikan APllet sitt /animes endepunkt.

Tilleggsfunksjonalitet: Vi har lagt til tekst på toppen av siden for å gi forklaring på at kortene er klikkbare og leder videre til en ny skjerm (DetailsScreen) der brukeren kan se utvidet informasjon om den valgte Animen.

AnimeDetailScreen



AnimeDetailScreen gir en mer detaljert visning av den valgte animen. Skjermen har en tilbake knapp indikert med pil som tar deg tilbake til animeScreen. Skjermen henter inn tittel, japansk tittel og utgivelsesår på lik måte som i animeScreen. Vi har i tillegg lagt til følgende verdier:

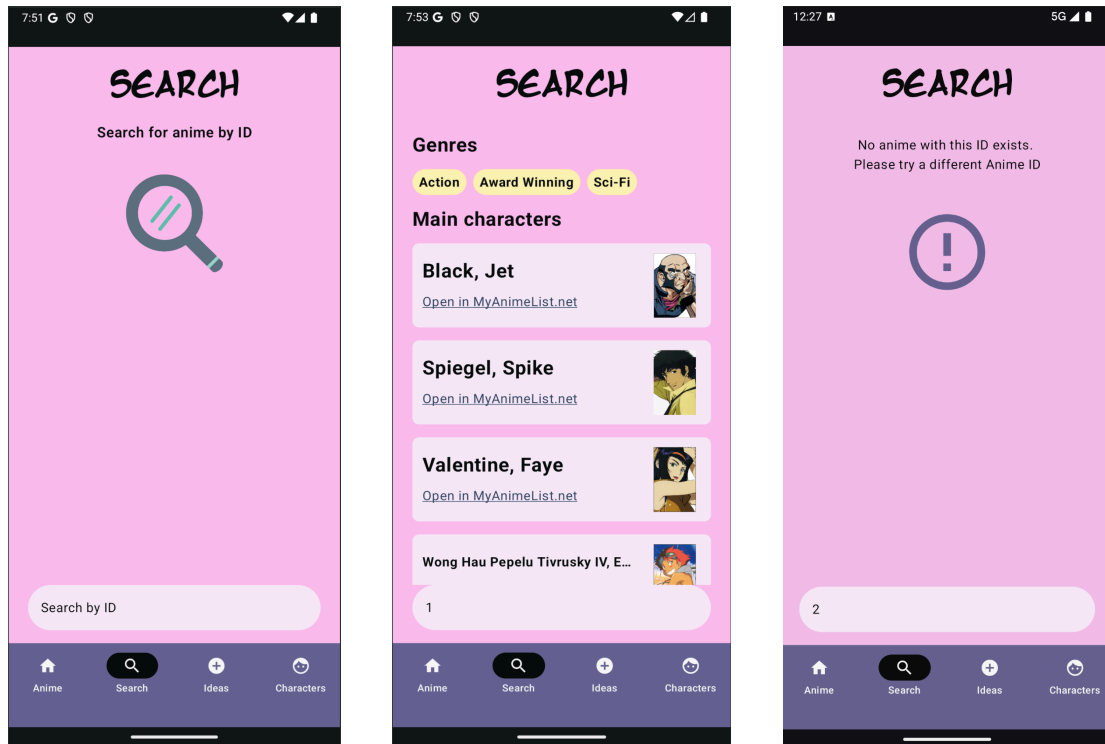
- **Synopsis** er hentet fra “synopsis” i APlet
- **Genre** printer ut stylede bokser for hver genre hentet fra “genres” listen i APlet.
- **Rating** vises som en stylet boks med ikon som henter inn ratingen til animen fra “score” i APlet.
- **Type** viser hva salgs type Anime det er (TV, Movie, Music, Special osv) og er hentet fra “type” i APlet. Dersom type er satt til TV vil det også vises verdi for antall episoder (Episodes) og for øvrige typer vil det vises verdi for varighet (Duration)

For å muliggjøre visningen av DetailsScreen ved klikk på et objekt i AnimeScreen, benytter vi oss av BackStackEntry for å hente ut nødvendige argumenter, som animeld, fra navigasjonsstakken. Dette gjør det mulig å vise spesifikke detaljer basert på brukerens valg, og sikrer at riktig data blir tilgjengelig for DetailScreen når den navigeres til.

Denne siden kaller på AnimeAPIRepository sin getAnimeById som returnerer et Anime objekt fra APlet sitt /anime/{id} ved hjelp av retrofit.

Tilleggsfunksjonalitet: Vi har lagt til en “More information” funksjonalitet der vi har lagt til en klikkbar link som tar deg videre til myanimelist.net sin side for den valgte animen. Verdien hentes fra “url” i APlet.

AnimeSearchScreen



Search skjermen (animeSearch) er en side der man kan søke på anime basert på id. Skjermen viser en forklarende tekst til brukeren og et søkefelt (TextField). Søkefeltet er plassert nederst på skjermen for enklere bruk på telefon. Vi har satt input verdien på søkefeltet til å kun ta imot tall for å unngå brukerfeil. En konsekvens av dette er at man ikke kan bruke ENTER tasten på PCen og må åpne “on-screen keyboard” på den virtuelle telefonen.

Vi har brukt forskjellige farger på “focused” og “unfocused” på både denne og alle andre skjermer for å gjøre det tydelig for brukeren hvilket inputfelt de skriver i.

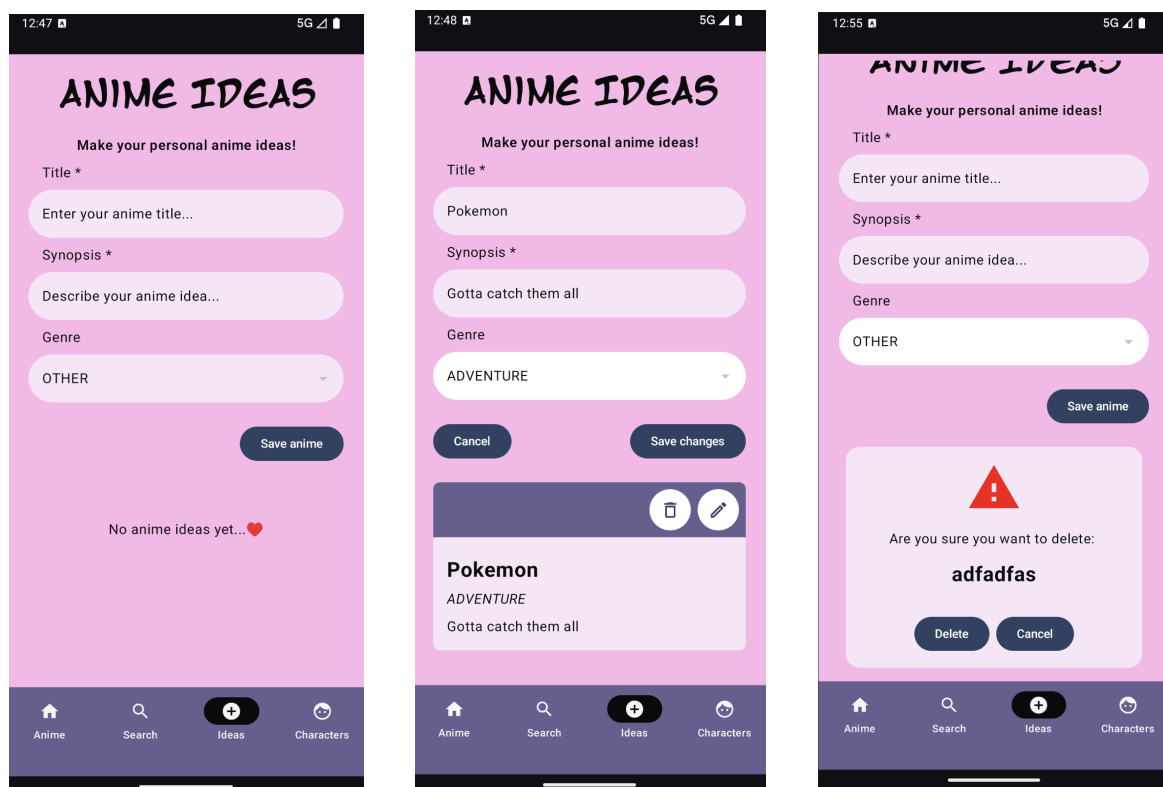
Hvis søket blir vellykket, vises en modifisert versjon av DetailScreen som også har hovedkarakterer fra den aktuelle animen. Her har vi brukt Coil for å vise bilder fra både Animen som er hentet og hovedkarakterene i animen

Vi har brukt retrofit for å hente både hovedkarakterene og animene som blir søkt på.

Tilleggsfunksjonalitet: Vi har lagt til en liste i DetailScreen som kun er aktiv når man er i SearchScreen som viser alle hovedkarakterer fra den aktuelle animen. Hovedkarakterene vises i bokser med tittel, navn og en link som leder til myanimelist.net siden for den aktuelle karakteren.

Hovedkarakterene er hentet fra APllet sitt `/anime/{id}/characters` endepunkt som viser alle karakterer fra en anime. Vi har valgt å filtrere karakterene vi viser i SearchScreen til å kun være hovedkarakterer. APllet har i utgangspunktet ikke et eget endepunkt for dette, så vi har filtrert på `"role" == main` i API repositoriet.

AnimelIdeasScreen



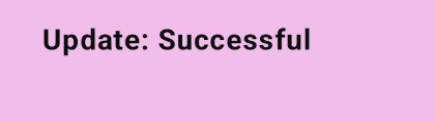
Anime Ideas-skjermen lar til å utvikle og lagre egne anime-ideer. Skjermen inneholder instruksjonstekst og tekstfelt for å samle inn nødvendig informasjon fra brukeren. Forklarende tekst og placeholder tekst for feltene tydeliggjør hva brukeren må fylle inn. Tittel og synopsis er obligatorisk, mens sjangerfeltet har en

standardverdi satt til "other". Brukeren lagrer ideen ved å trykke "save anime"-knappen, og dataene lagres i databasen med Room.

Hvis databasen er tom, vises "No anime ideas yet". Lagrede ideer vises som kort under inputfeltene, med knapper for sletting og redigering.

Ved å klikke på redigeringsknappen, oppdateres brukergrensesnittet til "redigeringsmodus", hvor tekstfeltene fylles med informasjonen fra den valgte ideen, slik at brukeren kan redigere ideen etter eget ønske. I denne modusen vises en "cancel"-knapp for å avslutte redigeringen, samt en "save changes"-knapp for å lagre eventuelle endringer.

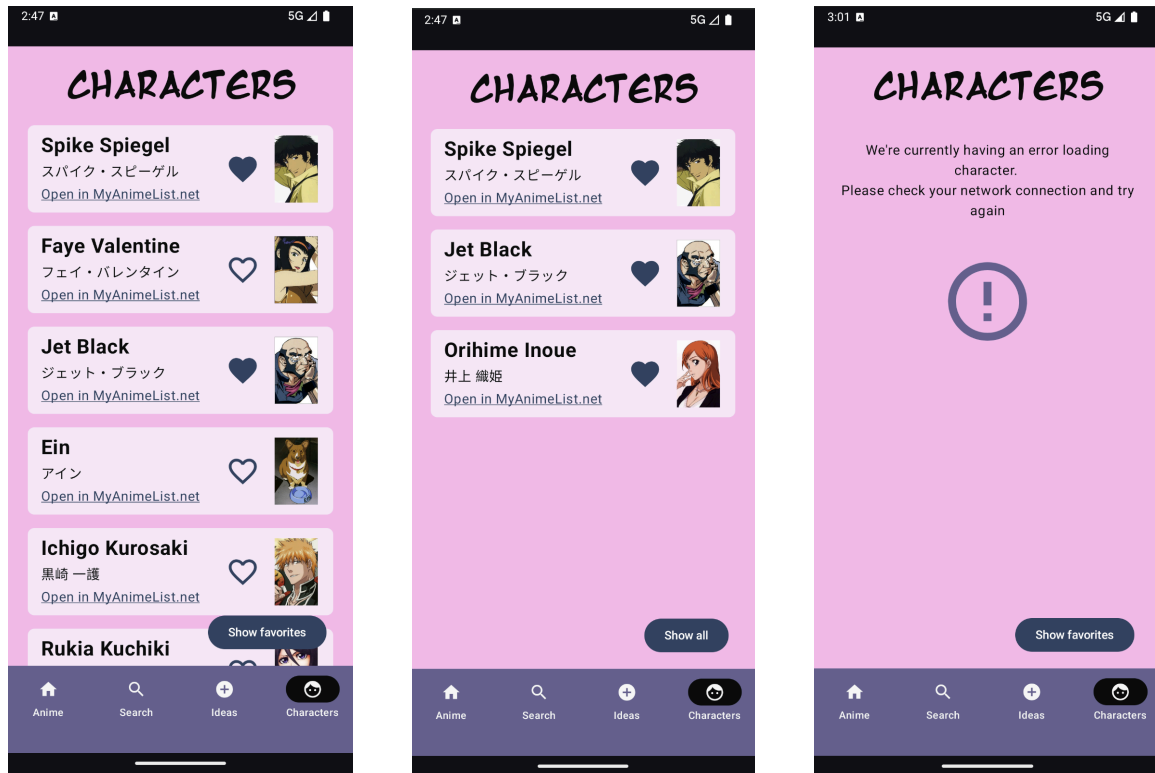
Når brukeren klikker på slett-knappen, aktiveres slettemodus. En advarselsboks vises for å forhindre utilsiktet sletting, hvor brukeren må bekrefte eller avbryte handlingen. Tittelen på ideen vises for å sikre at riktig idé er valgt, og gir ekstra trygghet mot feil. Etter hver databaseendring får brukeren en statusmelding som bekrefter om endringen var vellykket.



Update: Successful

Tilleggsfunksjonalitet: I stedet for å la brukeren skrive inn sjanger som fritekst, har vi valgt en mer strukturert løsning med en enum-klasse i databasen og en drop down meny på skjermen. Dette gjør valget enklere for brukeren og åpner for funksjoner som sjangerfiltrering senere. En ulempe er at enum-listen ikke dekker alle sjangre, men løsningen gir en mer konsistent og ryddig brukeropplevelse.

AnimeCharacterScreen



I Characters skjermen kan brukeren se karakterer som finnes i APllet. Dette er skjermen vi har laget som valgfri skjerm i henhold til oppgaven. Om datainnhenting fra APllet er vellykket, vises en liste med karakterkort på skjermen. Om datainnhenting mislykkes vil brukeren få opp en feilmelding som forteller.

Vi har brukt /characters endepunktet fra APllet på denne siden. I likhet med /anime endepunktet viser dette endepunktet kun en “page” med 25 objekter. Vi har valgt å holde oss til kun en page da APllet har totalt 211980 karakterer (Jikan API, u. å. -b) og innhenting av disse ville påvirket innlastingstiden på appen betydelig.

Hver karakter vises på et karakter kort som inneholder:

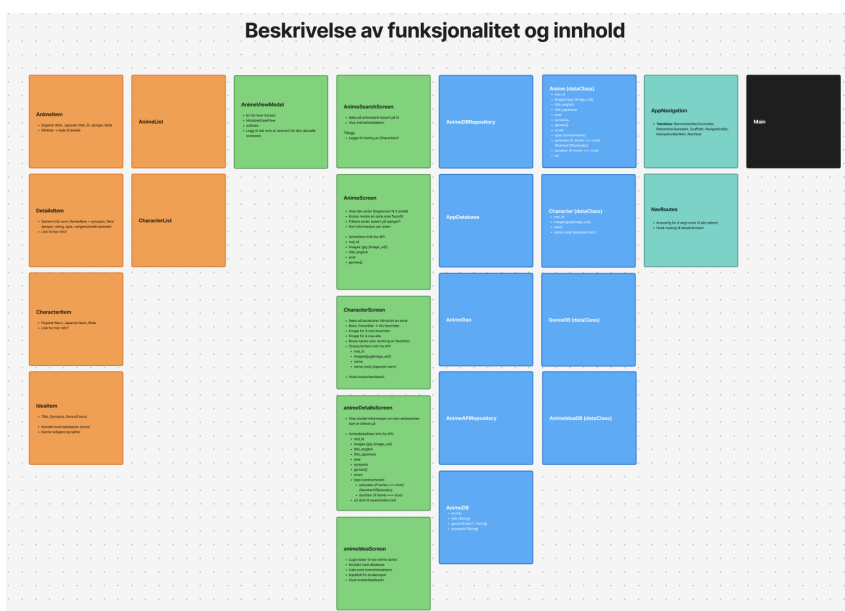
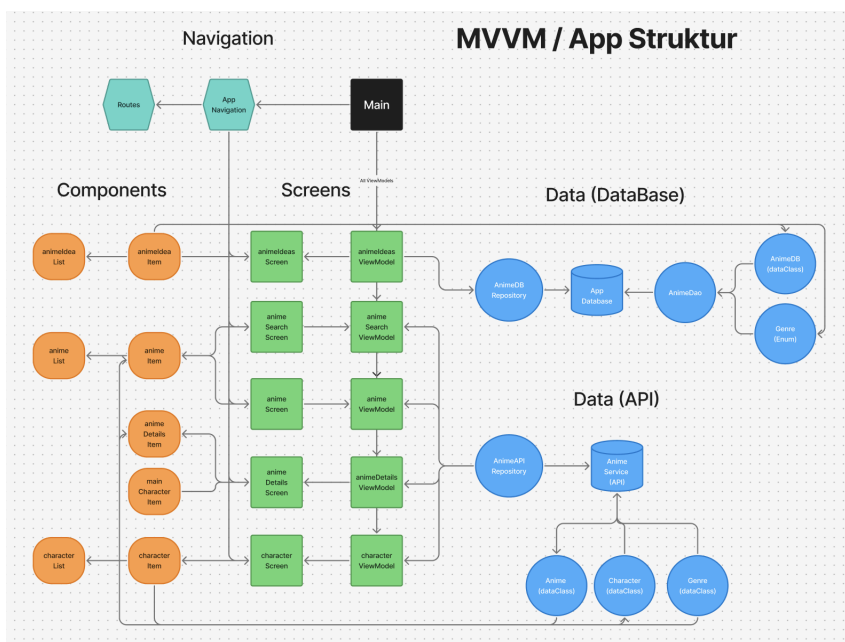
- Engelsk navn, er hentet fra name i APllet
- Japansk navn, hentet fra name_kanji i APllet
- Bilde, hentet fra “images” i APllet på samme måte som i AnimeScreen

- Link til myAnimeList.net som tar brukeren videre til myanimelist.net med utvidet informasjon om karakteren. Hentet fra "url" i APIet.

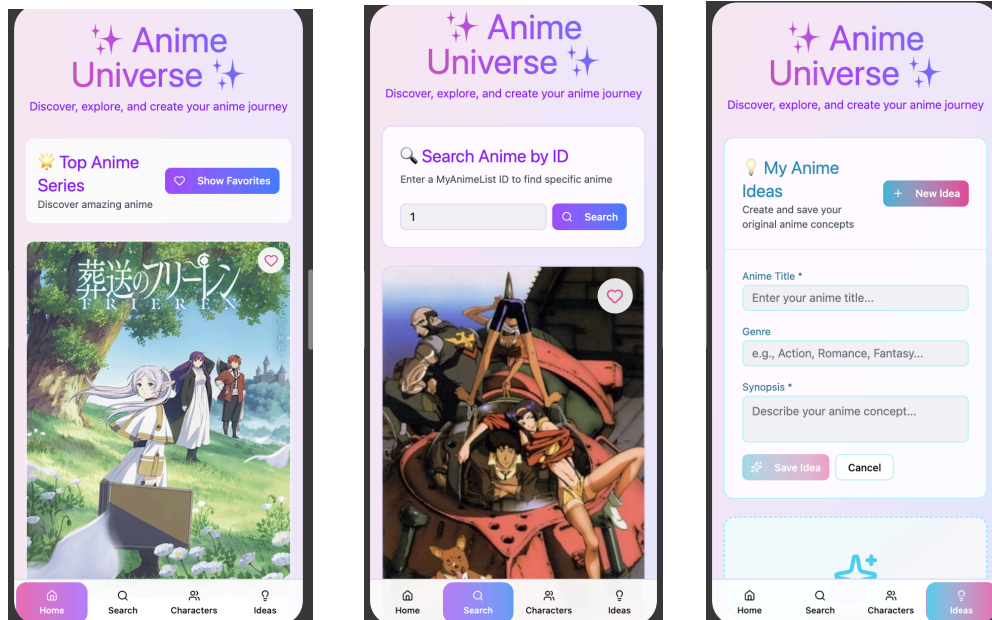
Tilleggsfunksjonalitet: Vi har implementert en favorittfunksjon som lar brukeren markere ønskede karakterer som favoritter. Hvert karakterkort har et klikkbart hjerte som fylles med farge når det aktiveres, for å indikere at karakteren er favorisert. Nederst på skjermen finnes en "Show favorites"-knapp som endrer brukergrensesnittet til kun å vise de favoriserte karakterene når den er klikket på. Når favorittene vises, endres knappen til "Show all", slik at brukeren enkelt kan gå tilbake til å se alle karakterene. Denne funksjonaliteten håndteres ved bruk av `mutableStateFlow` (`favoriteList`) i `ViewModel`, hvor vi har satt opp en `toggle`-funksjon for å sjekke om en karakter er merket som favoritt, samt legge til eller fjerne karakteren fra favorittlisten.

Del 2. Redegjørelse av planlegging og utvikling

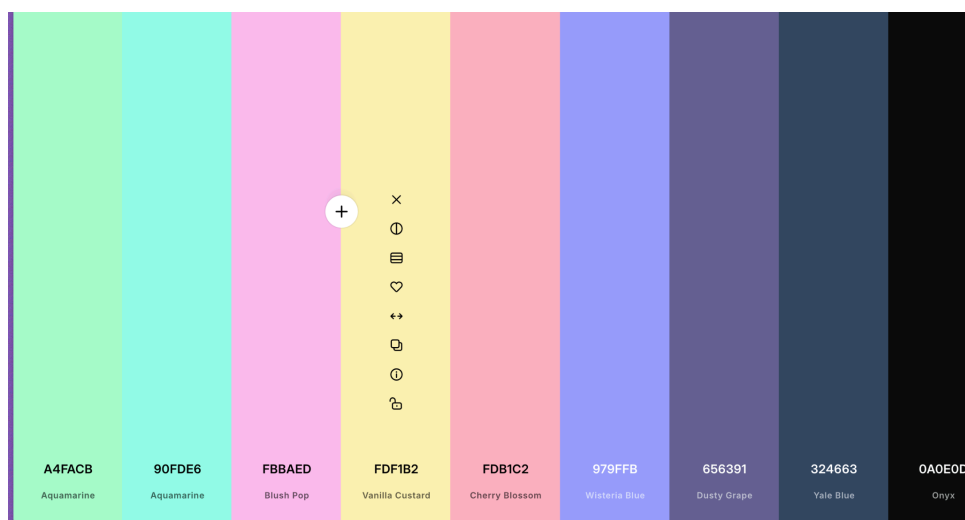
Vi startet prosjektet med å planlegge hvilke skjermer og funksjonalitet vi ønsket i appen. Dette ble kartlagt i FigJam, hvor vi satte opp forventet filstruktur og kommunikasjon mellom komponentene. Dette ga oss en felles forståelse av prosjektets struktur og gjorde det enklere å sette opp i AndroidStudio. Basert på Flow-skjemaet opprettet vi fargekodede "post-it lapper" som forklarte innholdet på de ulike skjermene, inkludert funksjonalitet og nødvendig data. Disse lappene ble brukt til å fordele arbeidsoppgaver og lage "to do"-lister for hver fil.



Etter å ha blitt enige om funksjonalitet og innhold, brukte vi FigmaMake til å lage en prototype av appen. Dette ga oss et visuelt inntrykk av fungerte som inspirasjon til design.



Videre satte vi retningslinjer for design og brukervennlighet. Vi valgte fonter og en fargepalett, og sjekket kontrasten mellom fargene med Coolor (Coolors, u. å.). Vi satte også standarder for skriftstørrelse for å sikre god lesbarhet (Wina Pradnyandari, 2023).



Vi benyttet GitHub for å kode sammen i prosjektet, noe som ga oss mulighet til å jobbe både remote og sammen. Vi hadde daglige møter der vi diskuterte oppgaver og mål. Denne arbeidsmåten ga alle gruppemedlemmene god oversikt over prosjektet og hverandres arbeid. Beslutninger ble tatt i fellesskap, og vi anser samarbeidet som vellykket.

Når vi ser tilbake, identifiserer vi områder for forbedring. Vi kunne ha jobbet komponentbasert tidligere og vært flinkere til å legge inn beskrivende kommentarer i koden fra start. Vi ser også forbedringspotensial i løsningen, som å legge til filtreringsfunksjoner på flere sider, gjøre GenreDropDown gjenbrukbar, og gi bedre feedback til brukeren på IdeasScreen. InputField på SearchScreen burde vært en komponent for fremtidig gjenbruk. Til tross for disse forbedringspunktene, er vi fornøyde med prosjektets utfall.

Kilder

Android Developers. (u. å.). *ViewModel overview*. Hentet 1. desember 2025 fra <https://developer.android.com/topic/libraries/architecture/viewmodel> Android Developers

Coolors. (u. å.). *Color Contrast Checker*. Hentet 1. desember 2025 fra <https://coolors.co/contrast-checker/112a46-acc8e5>

Jikan API. (u. å. -a). *Anime endpoint*. Hentet 1. desember 2025 fra <https://api.jikan.moe/v4/anime>

Jikan API. (u. å. -b). *Anime endpoint*. Hentet 1. desember 2025 fra <https://api.jikan.moe/v4/characters>

Wina Pradnyandari. (2023, 4. november). UI Typography Guidelines: iOS, Android, and Website font Size Guideline — Part 2. Medium. <https://medium.com/@wayanwina189/ui-typography-guidelines-ios-android-and-website-font-size-guideline-part-2-abcee14df8cb>